

Linux Production Shell Scripts

PART-3

Task Scheduler Shell Scripts

Here are several **Task Scheduler Shell Scripts** that help automate various tasks on Linux using cron jobs or a custom scheduling mechanism:

1. Simple Task Scheduler Script (Daily Task)

This script runs a specific task every day at a set time (e.g., 2:00 AM).

bash

Copy code

```
#!/bin/bash
```

```
# Task Scheduler: Daily Task Example
```

```
# Description: This script runs a task (e.g., a backup) daily at 2:00 AM.
```

```
TASK_LOG="/var/log/task_scheduler.log"
```

```
TASK_COMMAND="tar -czf /backup/my_backup.tar.gz  
/important_data"
```

```
log_task() {  
    TIMESTAMP=$(date "+%Y-%m-%d %H:%M:%S")  
    echo "$TIMESTAMP - Task started" >> "$TASK_LOG"  
    $TASK_COMMAND  
    echo "$TIMESTAMP - Task completed" >> "$TASK_LOG"  
}
```

log_task

Usage: Schedule the script using cron by adding this to your crontab (crontab -e):

bash

Copy code

```
0 2 * * * /path/to/your/script.sh
```

This will run the script at **2:00 AM every day**.

2. Hourly Task Scheduler (Log Cleanup)

This script runs a log cleanup every hour.

bash

Copy code

```
#!/bin/bash
```

```
# Task Scheduler: Hourly Log Cleanup
```

```
# Description: This script removes logs older than 7 days every hour.
```

```
LOG_DIR="/var/log/myapp"
```

```
LOG_FILE="/var/log/task_scheduler.log"
```

```
cleanup_logs() {  
    TIMESTAMP=$(date "+%Y-%m-%d %H:%M:%S")  
    echo "$TIMESTAMP - Starting log cleanup" >> "$LOG_FILE"  
    find $LOG_DIR -type f -name "*.log" -mtime +7 -exec rm {} \;  
    echo "$TIMESTAMP - Log cleanup completed" >> "$LOG_FILE"  
}
```

cleanup_logs

Usage: Schedule it to run hourly with the following cron job:

bash

Copy code

```
0 * * * * /path/to/your/script.sh
```

3. Weekly Backup Task Scheduler

This script runs a backup task every Sunday at 3:00 AM.

bash

Copy code

```
#!/bin/bash
```

```
# Task Scheduler: Weekly Backup Task
```

```
# Description: This script performs a backup every Sunday at 3:00 AM.
```

```
BACKUP_DIR="/home/user/backups"
```

```
SOURCE_DIR="/home/user/data"
```

```
BACKUP_LOG="/var/log/backup_task.log"
```

```
perform_backup() {
```

```
    TIMESTAMP=$(date "+%Y-%m-%d %H:%M:%S")
```

```
    echo "$TIMESTAMP - Starting backup" >> "$BACKUP_LOG"
```

```
    cp -r $SOURCE_DIR $BACKUP_DIR
```

```
    echo "$TIMESTAMP - Backup completed" >> "$BACKUP_LOG"
```

```
}
```

```
perform_backup
```

Usage: Schedule the script to run weekly (every Sunday at 3:00 AM):

bash

Copy code

```
0 3 * * SUN /path/to/your/script.sh
```

4. Monthly Disk Space Checker

This script checks disk space usage and sends a report if usage exceeds 90%.

bash

Copy code

```
#!/bin/bash
```

```
# Task Scheduler: Monthly Disk Space Check
```

```
# Description: This script checks disk space usage every month and  
sends an alert if it's over 90%.
```

```
THRESHOLD=90
```

```
ALERT_EMAIL="admin@example.com"
```

```
DISK_USAGE=$(df / | grep / | awk '{ print $5 }' | sed 's/%//g')
```

```
if [ "$DISK_USAGE" -gt "$THRESHOLD" ]; then
```

```
    echo "Disk space is critically high: ${DISK_USAGE}% usage" |  
mail -s "Disk Space Alert" "$ALERT_EMAIL"
```

```
    echo "$(date '+%Y-%m-%d %H:%M:%S') - Disk Space Alert:  
${DISK_USAGE}% usage" >> /var/log/disk_space.log
```

```
fi
```

Usage: Schedule it to run monthly:

```
bash
```

Copy code

```
0 0 1 * * /path/to/your/script.sh
```

5. Task Scheduler with Custom Interval (Every 15 Minutes)

This script runs a specific task every 15 minutes.

```
bash
```

Copy code

```
#!/bin/bash
```

```
# Task Scheduler: Task Every 15 Minutes
```

```
# Description: This script runs a custom task every 15 minutes.
```

```
TASK_LOG="/var/log/task_every_15min.log"
```

```
TASK_COMMAND="python3 /path/to/script.py"
```

```
log_task() {
```

```
    TIMESTAMP=$(date "+%Y-%m-%d %H:%M:%S")
    echo "$TIMESTAMP - Task started" >> "$TASK_LOG"
    $TASK_COMMAND
    echo "$TIMESTAMP - Task completed" >> "$TASK_LOG"
}
```

log_task

Usage: Schedule the script to run every 15 minutes:

bash

Copy code

```
*/15 * * * * /path/to/your/script.sh
```

6. Task Scheduler with Random Time Interval

This script runs a task at a random interval between 1 and 10 minutes.

bash

Copy code

```
#!/bin/bash
```

```
# Task Scheduler: Random Time Interval
```

```
# Description: This script runs a task at a random interval between 1
and 10 minutes.
```

```
TASK_LOG="/var/log/random_task.log"
```

```
TASK_COMMAND="echo 'Random task executed'"
```

```
run_random_task() {
```

```
    TIMESTAMP=$(date "+%Y-%m-%d %H:%M:%S")
```

```
    echo "$TIMESTAMP - Task started" >> "$TASK_LOG"
```

```
    $TASK_COMMAND
```

```
    echo "$TIMESTAMP - Task completed" >> "$TASK_LOG"
```

```
}
```

```
RANDOM_INTERVAL=$(( ( RANDOM % 10 ) + 1 )) # Random
between 1 and 10 minutes
sleep $((RANDOM_INTERVAL * 60)) # Sleep for random minutes
```

```
run_random_task
```

Usage: You can run this script using a simple cron job that keeps executing it:

```
bash
Copy code
* * * * * /path/to/your/script.sh
```

7. Task Scheduler: System Health Check

This script checks the system's health and logs any critical issues (CPU, memory, disk usage).

```
bash
Copy code
#!/bin/bash
```

```
# Task Scheduler: System Health Check
# Description: This script checks the system's CPU, memory, and
disk usage, and logs any issues.
```

```
HEALTH_LOG="/var/log/system_health.log"
```

```
check_system_health() {
    CPU_USAGE=$(top -bn1 | grep "Cpu(s)" | sed "s/.*, *\([0-9.]*\)%"*
id.*\1/" | awk '{print 100 - $1}')
    MEM_USAGE=$(free -m | grep Mem | awk '{print $3/$2 * 100.0}')
    DISK_USAGE=$(df / | grep / | awk '{ print $5 }' | sed 's/%//g')

    TIMESTAMP=$(date "+%Y-%m-%d %H:%M:%S")
    echo "$TIMESTAMP - CPU Usage: $CPU_USAGE%, Memory
Usage: $MEM_USAGE%, Disk Usage: $DISK_USAGE%" >>
"$HEALTH_LOG"
```

if [\$(echo "\$CPU_USAGE > 90" | bc) Here are several **Task Scheduler Shell Scripts** that help automate various tasks on Linux using cron jobs or a custom scheduling mechanism:

These **Task Scheduler Shell Scripts** automate important system management tasks, from backups to system health checks, ensuring smooth operations with minimal manual intervention. You can use cron jobs to schedule them as per your needs.

System Information Shell Scripts

1. System Information Overview Script

This script provides an overview of your system, including the hostname, OS, kernel version, uptime, and CPU load.

```
bash
Copy code
#!/bin/bash

# System Information Overview
echo "System Overview:"
echo "=====
echo "Hostname: $(hostname)"
echo "OS: $(uname -o)"
echo "Kernel Version: $(uname -r)"
echo "Uptime: $(uptime -p)"
echo "CPU Load: $(uptime | awk -F'load average:' '{ print $2 }')"
echo "Disk Usage: $(df -h / | awk 'NR==2 {print $3 "/" $2 " (" $5 "
used)}")"
```

Usage: This will output the basic system information for a quick overview.

2. CPU Information Script

This script displays detailed information about the CPU, including its model, cores, and clock speed.

bash

Copy code

#!/bin/bash

CPU Information

echo "CPU Information:"

echo "=====

lscpu | grep -E 'Model name|CPU(s)|Thread|Core|Socket'

echo "CPU Usage: \$(top -bn1 | grep "Cpu(s)" | sed "s/.*, *\([0-9.]*\)%" id.*\1/" | awk '{print 100 - \$1 "%"}")"

Usage: This will provide detailed CPU information and usage.

3. Memory Usage Script

This script shows the total, used, and free memory on the system.

bash

Copy code

#!/bin/bash

Memory Usage

echo "Memory Information:"

echo "=====

free -h | awk 'NR==2 {print "Total: "\$2 " Used: "\$3 " Free: "\$4 "

Buffers/Cache: "\$6}'

Usage: This script will display memory usage statistics in a human-readable format.

4. Disk Usage Script

This script checks the disk usage of all mounted filesystems.

bash

Copy code

```
#!/bin/bash
```

```
# Disk Usage
```

```
echo "Disk Usage Information:"
```

```
echo "=====
```

```
df -h | grep -vE '^Filesystem|tmpfs|cdrom' # Exclude tmp and  
cdrom
```

Usage: This will output disk space usage for all mounted filesystems, excluding temporary filesystems and CD-ROMs.

5. Network Information Script

This script shows basic network information including IP addresses, netmask, and default gateway.

bash

Copy code

```
#!/bin/bash
```

```
# Network Information
```

```
echo "Network Information:"
```

```
echo "=====
```

```
echo "IP Address: $(hostname -I)"
```

```
echo "Netmask: $(ifconfig | grep -w inet | awk '{print $4}')
```

```
echo "Gateway: $(ip route | grep default | awk '{print $3}')
```

Usage: This provides basic network configuration and connection details.

6. System Resource Usage Script

This script provides a snapshot of resource usage, including CPU, memory, disk, and swap.

bash

Copy code

```
#!/bin/bash
```

System Resource Usage

```
echo "System Resource Usage:"
```

```
echo "=====
```

```
echo "CPU Usage: $(top -bn1 | grep "Cpu(s)" | sed "s/./, */([0-9.]*)%* id.*\1/" | awk '{print 100 - $1 "%"}')
```

```
echo "Memory Usage: $(free -h | grep Mem | awk '{print $3 "/" $2}')
```

```
echo "Disk Usage: $(df -h / | awk 'NR==2 {print $3 "/" $2 " (" $5 "
used)"}')"
```

```
echo "Swap Usage: $(free -h | grep Swap | awk '{print $3 "/" $2}')
```

Usage: This will output the usage of all critical system resources.

7. Uptime & Load Average Script

This script checks the system uptime and load average for the last 1, 5, and 15 minutes.

bash

Copy code

```
#!/bin/bash
```

System Uptime & Load Average

```
echo "System Uptime and Load Average:"
```

```
echo "====="
```

```
uptime | awk -F'(|,:)+ ' '{ print "Uptime: " $3 " days " $4 " hours " $5 "
minutes"; print "Load Average: 1 min: "$10 " 5 min: "$11 " 15 min:
"$12 }
```

Usage: This provides the system uptime and load averages, helping to monitor the system's health.

8. Top 5 Memory Consuming Processes

This script shows the top 5 processes consuming the most memory.

```
bash
Copy code
#!/bin/bash

# Top 5 Memory Consuming Processes
echo "Top 5 Memory Consuming Processes:"
echo "=====
ps aux --sort=-%mem | awk 'NR<=6 {print $1, $3, $11}' # Skip
header and show top 5
```

Usage: This will display the processes consuming the most memory in your system.

These **System Information Shell Scripts** are perfect for monitoring your system's health and quickly gathering key stats. You can schedule these scripts to run periodically using cron or use them manually for on-demand insights into your system's performance.

Disk Space Monitoring Shell Scripts

Here are some **Disk Space Monitoring Shell Scripts** that can help you track the available space on your system and send alerts when disk usage exceeds a certain threshold.

1. Basic Disk Space Monitoring Script

This script checks the disk space usage and reports if any filesystem exceeds the 90% usage threshold.

bash

Copy code

#!/bin/bash

Disk Space Monitoring

Description: This script checks disk usage and sends an alert if it exceeds 90%.

THRESHOLD=90

ALERT_EMAIL="admin@example.com"

DISK_USAGE=\$(df / | grep / | awk '{ print \$5 }' | sed 's/%//g')

if ["\$DISK_USAGE" -gt "\$THRESHOLD"]; then

echo "Disk space usage is over the threshold! Current usage: \${DISK_USAGE}%" | mail -s "Disk Space Alert" "\$ALERT_EMAIL"

echo "\$(date "+%Y-%m-%d %H:%M:%S") - Disk space alert: Usage is \${DISK_USAGE}%" >> /var/log/disk_space_monitor.log
else

echo "\$(date "+%Y-%m-%d %H:%M:%S") - Disk space is under control: Usage is \${DISK_USAGE}%" >>

/var/log/disk_space_monitor.log

fi

Usage: Schedule this script using cron to run at regular intervals (e.g., every hour).

bash

Copy code

0 * * * * /path/to/disk_space_monitor.sh

2. Disk Space Usage Monitoring for All Filesystems

This script checks the usage of all mounted filesystems and sends an email alert if any of them exceed the threshold.

bash
Copy code
#!/bin/bash

Disk Space Monitoring for All Filesystems
Description: This script checks the usage of all filesystems and alerts if any exceed 90%.

THRESHOLD=90
ALERT_EMAIL="admin@example.com"

```
df -h | grep -vE '^Filesystem|tmpfs|cdrom' | while read -r line; do
    FILESYSTEM=$(echo $line | awk '{print $1}')
    USAGE=$(echo $line | awk '{print $5}' | sed 's/%//')

    if [ "$USAGE" -gt "$THRESHOLD" ]; then
        echo "Warning: $FILESYSTEM is over ${THRESHOLD}% full!
Current usage: ${USAGE}%" | mail -s "$FILESYSTEM - Disk Space
Alert" "$ALERT_EMAIL"
        echo "$(date "+%Y-%m-%d %H:%M:%S") - ALERT:
$FILESYSTEM usage is ${USAGE}%" >>
/var/log/disk_space_monitor.log
    fi
done
```

Usage: This can be scheduled using cron to run periodically.

bash
Copy code
0 * * * * /path/to/disk_space_monitor_all.sh

3. Disk Space Usage and Report Script

This script not only checks disk space usage but also generates a report with disk usage details and sends it via email.

bash
Copy code

```
#!/bin/bash
```

```
# Disk Space Report
```

```
# Description: This script generates a disk space report and emails  
it if any filesystem exceeds 90% usage.
```

```
THRESHOLD=90
```

```
ALERT_EMAIL="admin@example.com"
```

```
REPORT_FILE="/tmp/disk_space_report.txt"
```

```
# Generate Report
```

```
echo "Disk Space Usage Report - $(date)" > $REPORT_FILE
```

```
echo "===== " >> $REPORT_FILE
```

```
df -h | grep -vE '^Filesystem|tmpfs|cdrom' >> $REPORT_FILE
```

```
# Check for usage over threshold
```

```
df -h | grep -vE '^Filesystem|tmpfs|cdrom' | while read -r line; do
```

```
    USAGE=$(echo $line | awk '{print $5}' | sed 's/%//g')
```

```
    if [ "$USAGE" -gt "$THRESHOLD" ]; then
```

```
        echo "ALERT: Disk usage exceeds $THRESHOLD% on  
$(echo $line | awk '{print $1}')) with $USAGE% usage." | mail -s
```

```
"Disk Space Alert: $(hostname)" "$ALERT_EMAIL"
```

```
        echo "$(date "+%Y-%m-%d %H:%M:%S") - ALERT: $(echo  
$line | awk '{print $1}')) usage is ${USAGE}%" >>
```

```
/var/log/disk_space_monitor.log
```

```
    fi
```

```
done
```

```
# Send Report
```

```
cat $REPORT_FILE | mail -s "Disk Space Report for $(hostname)"  
"$ALERT_EMAIL"
```

```
rm $REPORT_FILE
```

Usage: Schedule this script to run daily at midnight to generate daily disk space usage reports.

```
bash
```

```
Copy code
```

```
0 0 * * * /path/to/disk_space_report.sh
```

4. Disk Space Monitoring with Notification

This script monitors the disk space and sends a notification via notify-send (for desktop environments) if the space exceeds the defined threshold.

bash

Copy code

```
#!/bin/bash
```

```
# Disk Space Monitoring with Notification
```

```
# Description: This script checks disk usage and provides  
notifications when space exceeds 90%.
```

```
THRESHOLD=90
```

```
DISK_USAGE=$(df / | grep / | awk '{ print $5 }' | sed 's/%//g')
```

```
if [ "$DISK_USAGE" -gt "$THRESHOLD" ]; then
```

```
    MESSAGE="Warning: Disk space usage is at  
${DISK_USAGE}%"
```

```
    notify-send "Disk Space Alert" "$MESSAGE"
```

```
    echo "$(date "+%Y-%m-%d %H:%M:%S") - $MESSAGE" >>  
/var/log/disk_space_monitor.log  
fi
```

Usage: This is suitable for systems with a graphical interface. You can also schedule it using cron.

bash

Copy code

```
*/30 * * * * /path/to/disk_space_notify.sh
```

5. Disk Space Check with Email Alerts and Log

This script checks disk space usage and logs it to a file. If the usage exceeds the threshold, it sends an email alert.

```
bash
Copy code
#!/bin/bash
```

```
# Disk Space Check and Alert
# Description: This script checks disk space usage, logs it, and
alerts if the usage exceeds 90%.
```

```
THRESHOLD=90
ALERT_EMAIL="admin@example.com"
LOG_FILE="/var/log/disk_space_alert.log"
```

```
# Get current disk usage
DISK_USAGE=$(df / | grep / | awk '{ print $5 }' | sed 's/%//g')
```

```
# Check if usage exceeds threshold
if [ "$DISK_USAGE" -gt "$THRESHOLD" ]; then
    echo "$(date "+%Y-%m-%d %H:%M:%S") - Disk space is over
$THRESHOLD% used. Current usage: ${DISK_USAGE}%" >>
$LOG_FILE
    echo "Disk space alert: Current usage is ${DISK_USAGE}%" |
mail -s "Disk Space Alert for $(hostname)" "$ALERT_EMAIL"
fi
```

Usage: Schedule the script to run at regular intervals (e.g., every hour).

```
bash
Copy code
0 * * * * /path/to/disk_space_check.sh
```

These **Disk Space Monitoring Shell Scripts** can be helpful to track and manage disk usage, ensuring your systems don't run out of space unexpectedly. You can customize the threshold and alerting mechanisms based on your needs.

**FOLLOW PRINCE JOSHI FOR FURTHER CONTENT
LIKE THIS IN FUTURE**

PRINCE JOSHI